

Quant Methods in Finance I - Final Project

Stuff vs Command: Decomposing Contact Quality in MLB Pitch-Level Statcast Data

Sean Nicgorski

October 1, 2026

Introduction

Evaluating an athlete's true ability or what makes them great is one of the oldest problems in sports analytics. For baseball, traditional pitching statistics like ERA and win-loss record are heavily influenced by factors out of the pitcher's control like defense, park factor, or just batter luck/skill. In recent years, the MLB's statcast system has made it possible to isolate the "stuff" a pitcher has. "Stuff" refers to only the pure physical characteristics of each pitch thrown, velocity, movement, shape, etc. So what "stuff" limits the true quality of contact in the MLB?

To answer this question using the ML techniques we have worked on in class, I will be fitting a model on all MLB PA-ending pitches from 2024-2026 then apply the findings to evaluate the 2026 White Sox staff as a true out-of-sample test. Our target variable will be Statcast's xwOBA (expected weighted on-base average) metric.

xwOBA: wOBA is a run-value based hitting metric calculated from past run-scoring environments that assigns a weight to an outcome proportional to how many runs it's worth. It recognizes that extra base hits contribute far more than a single. A strikeout is worth **0**, a walk or HBP is worth **0.69–0.73**, a single is worth **0.88**, a double **1.24**, a triple **1.56**, and a home run **2.00**. Statcast's expected (xwOBA) version of this metric estimates the wOBA based on the value a batted ball "should" produce from its quality of contact (velocity, launch angle).

My model will have statcast's physical only pitch characteristics (velocity, spin, release, location, vertical/horizontal movement, etc.) as inputs and use them to predict xwOBA on a pitch.

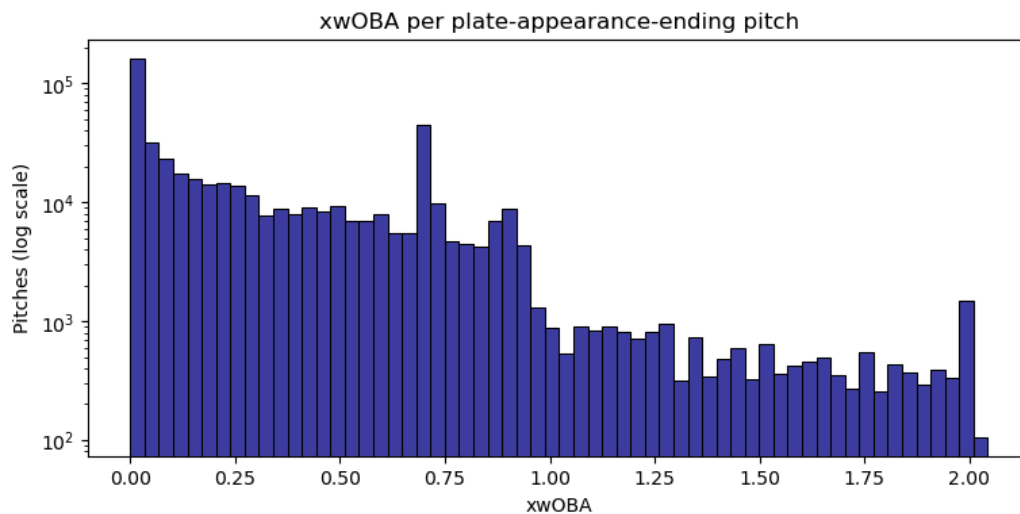
Related Work

This isn't a new category of question in baseball. Pitch quality statistics have become a common tool in recent years. Notably, Stuff+ by Fangraphs which uses physical pitch characteristics to predict an expected run value of a pitch independent of the actual outcome. What makes my model different is the decomposition of the full range of statcast data vs stuff-only (This was not intentional initially). The two-model approach will separate raw pitch quality from command

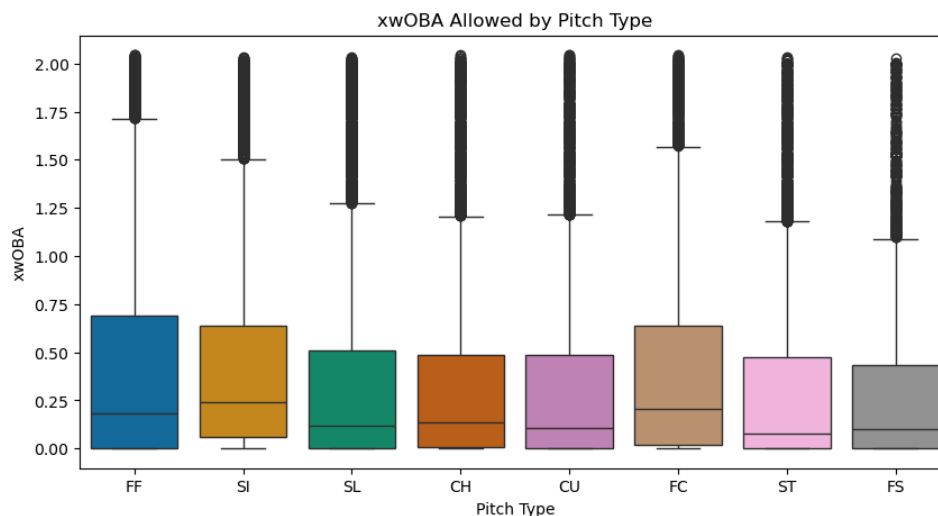
which during the project I found did not produce a very valuable result. In my evaluation of the 2026 White Sox staff, I took inspiration from Fangraphs and called the normalized score for each pitcher I calculated from my models Stuff+ and Full+. It sets 100 to be league average and 110 being 10% above average. While similar in name and style to Fangraphs Stuff+, the important distinction is that it is calculated on xwOBA rather than Expected Run Value.

Data Description

Data comes from a raw pull on the `pybaseball.statcast()` package from 2024-04-01 to 2026-09-28 and filters for the regular season only. The raw pull gives us over 2.2m pitches, we filter for the main 8 pitch types and only pitches with valid xwOBA data (PA ending pitches) which brings the total down to ~540k rows. After we drop any row with null values and filter for pitchers who appear 200+ times, we get 496,489 pitches from 687 pitchers.



Pitch type counts are as follows: FF 151,731 / SI 87,315 / SL 71,132 / CH 55,700 / CU 37,905 / FC 37,870 / ST 37,390 / FS 17,446.



I did some feature engineering as well. First, I normalized movement, release, and location for both right-handed and left-handed pitchers so arm-side break is positive for both rather than being opposite. Second, I did some generalizations on pitch types, classify a slider and slurve the same, knuckle-curve and curve the same. Lastly, I added a dummy variable for R/R or L/L match ups and one for being a strike.

To finish up, I pulled the 2026 White Sox out of the sample completely to train/validate/test on data that didn't include them at all so the analysis at the end is a true application of the model.

Methods

The main ML method we use is XGBoost. This model uses shallow trees that fit the residuals of older trees, it is a bunch of small simple decision trees where each new tree just tries to fix the mistakes of trees before it. I picked this method over a simpler regression because pitching data is full of interactions that would be messy to write out for regression, for example high velocity is good for a fastball but maybe not for a change up. The trees in XGBoost do this naturally, they split data by pitch type then figure out what matters within as the tree expands.

The strengths of the XGBoost method are that, in addition to handling interactions automatically, it is pretty resistant to overfitting. Since single pitch data on xwOBA can be noisy, good pitches can be hit great and you can get away with bad pitches, that was very important and why I kept the trees shallow (max_depth=6) and learning rate low (0.05). Besides those two, other

parameters were just set at what was recommended during my research. We used RMSE to measure how far off the predictions are from the actual xwOBA values on average. The main weakness of this model is that it's a black box. You can't look at the model and see what predictors are meaningful so I had to use SHAP to supplement that analysis. SHAP decomposes each feature into additive contributions, similar to a linear regression's coefficient number except it is calculated after the prediction is made and it not positive or negative

Results & Discussion

The data had a baseline RMSE of 0.3672, that is from just guessing the average xwOBA.

Full Model: Our full model (including location) had a RMSE on the test set of 0.3508 and R^2 of 0.0848. This looked good but when doing the SHAP analysis it was clear that location metrics were dominating the predictions. Horizontal and Vertical location combined for nearly double the predictive power of the next highest feature. The model definitely wasn't wrong, pitcher command is insanely important but this just isn't much of a new insight. To not just measure command, I decided to run a second model with the exact same parameters but remove the location features.

Stuff-Only Model: I removed location features and ran the exact same model. The RMSE comes out to 0.3638 and R^2 to 0.0157, this may seem bad at first. However, doing further analysis, the stuff-only model redeemed itself. The model's average predictions compared to the actual average values for 10 even/ordered bins was only off by 0.01 or less in each instance. This stuff-only model can accurately guess which pitches are good and bad on average which is what matters far more than predicting single PA results from their final pitch. To add onto that, we can look at pitcher level aggregation of this model's results. Results in table below:

Model	Pitch-level R^2	Pitcher-level r	Pitcher-level r^2
Full (stuff + location)	0.0848	0.463	0.214
Stuff only	0.0157	0.383	0.147
Stuff as share of full	18%	n/a	69%

The results here justify looking at pitchers' pitch characteristics independent of location despite the extremely low R^2 . This stuff model is a much weaker predictor of what happens in any given at-bat but is meaningfully strong at evaluating a pitch or pitcher's value over time based on his stuff which is what matters for us to apply this framework.

Below is the analysis of predicted xwOBA values by each of our models, actual xwOBA, actual wOBA, and number of result pitches for pitchers who have recorded 50+ outcome pitches in 2026 for the Chicago White Sox, luck is wOBA - xwOBA, command is stuff - full:

Player Name	n	Predict (Stuff Model)	Predict (Full Model)	Statcast xwOBA	wOBA	Command	Luck	Stuff+	Full+
Taylor, Grant	335	0.269	0.273	0.267	0.263	-0.005	-0.004	114	112
Newcomb, Sean	366	0.324	0.338	0.318	0.291	-0.014	-0.027	96	92
Richards, Trevor	320	0.347	0.335	0.312	0.299	0.013	-0.013	88	93
Burke, Sean	712	0.31	0.309	0.3	0.306	0.001	0.006	100	101
Hicks, Jordan	207	0.285	0.28	0.26	0.313	0.006	0.053	108	110
Domínguez, Seranthony	160	0.297	0.305	0.328	0.314	-0.008	-0.014	104	102
Hudson, Bryan	273	0.324	0.331	0.278	0.316	-0.007	0.038	96	94
Martin, Davis	592	0.325	0.34	0.329	0.325	-0.015	-0.004	95	91
Fedde, Erick	601	0.312	0.308	0.334	0.33	0.004	-0.004	100	101
Leasure, Jordan	75	0.298	0.279	0.417	0.343	0.02	-0.074	104	110
Schultz, Noah	319	0.33	0.342	0.358	0.344	-0.012	-0.014	94	90
Murphy, Chris	150	0.3	0.342	0.303	0.346	-0.042	0.044	104	90
Eisert, Brandon	115	0.313	0.307	0.288	0.35	0.006	0.061	99	101

White Sox Takeaways: The staff is slightly below average overall and command issues make them marginally worse. Grant Taylor is the clear standout with elite grades from both models. Chris Murphy has the most actionable case. His stuff grades above average at 104 but the full grade falls to 90. There is a command problem sitting on top of a usable arsenal which is much easier to fix than a lack of stuff, Brazoban shows a similar but more severe pattern but his sample size is small. Trevor Richards is the most curious case. He grades below average on both models but ranks third in wOBA and seventh in xwOBA allowed on the team. So he has found some success despite his underlying metrics being unimpressive. The White Sox should be cautious with him in the future despite decent results this year.

Another application is that we can use the stuff-model to predict the quality of each pitcher's individual pitches and look for big differences in quality and usage. For example, for Luis Castillo; his change-up grades as his best pitch yet he only throws it 23% of the time compared to 36% for the fastball and 29% for the sinker. There could be explanations (sequencing, location issues) that perfectly explain this but it's worth looking into for sure.

Conclusion & Future Work

Stuff explains a small amount of pitch-level outcomes but a majority (69%) comes from pitcher-level differences. When we separate stuff from command we can see specific and actionable findings for MLB pitchers which teams can incorporate into game decisions, team management, and practice plans. Next, it would be nice to make the model more complex using situations (count, handedness matchup, players on-base), Fangraphs is able to grade every pitch because they use run-value as the target which doesn't require an outcome pitch. Overall, it was fun to try to make an alternative Stuff+ model on my own and will definitely consider adding onto it in the future, likely using the same ML techniques.

Contribution

I worked on this project alone.

Bibliography

Petti, B., et al. pybaseball: a Python package for baseball data analysis. Open-source, GitHub.
<https://github.com/jldbc/pybaseball>

Baseball Savant. "Statcast Search CSV Documentation." MLB Advanced Media.
<https://baseballsavant.mlb.com/csv-docs>

MLB.com. "Statcast." MLB Glossary. <https://www.mlb.com/glossary/statcast>

McGrattan, O. (2023, March 10). Stuff+, Location+, and Pitching+ primer. FanGraphs
Sabermetrics Library. <https://library.fangraphs.com/pitching/stuff-location-and-pitching-primer/>

Swartz, P., Grosskopf, M., Bingham, D., & Swartz, T. B. (2017). The Quality of Pitches in Major
League Baseball. The American Statistician, 71(2), 148–154.
<https://doi.org/10.1080/00031305.2016.1264313>